

CCPS616 Lab 2 – United We Stand, Divided We Conquer

Preamble

In this lab you will solve problems using an iterative linear approach, and then a divide and conquer approach. You will implement your algorithms and verify their correctness against each other. Your implementations must be from the ground up without linking to any third-party libraries. Your lab should be completed in a single cpp file called **dnc.cpp**, and should compile at the command line very simply as follows:

```
g++ -o lab2 dnc.cpp lab2.cpp
```

Lab Description

The problems you will solve are described below. You will implement solutions for each in the function stubs provided in the dnc.cpp file. You will solve each problem two different ways. First using an iterative approach, and then using a recursive divide and conquer approach.

Part I) First Missing Integer

Propose and implement two algorithms for finding the first missing value in a sorted array of integers. All elements of the array are non-negative (greater than or equal to zero), and there are no duplicated values. For example, if the input was { 0, 1, 2, 3, 5, 6, 8, 9 }, then the first missing integer would be 4.

```
int first_missing_iterative(const std::vector<int> & sorted_vec)
```

Your first approach should run in **O(n)** and be implemented iteratively. In a comment above your implementation, briefly explain why your function runs in O(n) time.

```
int first_missing_dnc(const std::vector<int> & sorted_vec)
```

Your divide and conquer algorithm must run in **O(logn)**. In a comment above your divide and conquer implementation, briefly explain why your function runs in logarithmic time.

Part II) Counting Inversions

Propose and implement two algorithms for counting the number of inversions in an array. An inversion occurs when two elements in the array are out of order (refer to insertion sort discussion from lecture #3 for more information).

Hint: In both the iterative and divide and conquer approaches, think about how this problem could be solved using modified sorting algorithms...

```
int count_inversions_iterative(const std::vector<int> & arr)
```

This function must be implemented iteratively and run in **$O(n^2)$** . In a comment above your iterative implementation, briefly explain why your function runs in $O(n^2)$ time.

```
int count_inversions_dnc(const std::vector<int> & arr)
```

Your divide and conquer algorithm must run in **$O(n \log n)$** . In a comment above your divide and conquer implementation, briefly explain why your function runs in $O(n \log n)$ time.

Part III) Maximum Subarray Sum

We covered this one in class. Implement two algorithms for finding the maximum subarray sum. For example, the array `{ -12, 1, 4, -3, 20, -1, -10, -2 }` has a maximum subarray sum of 22 (bolded). Come up with your own iterative implementation (yes it will be inefficient), and then implement the divide and conquer approach we discussed in class.

```
int max_subarray_sum_iterative(const std::vector<int> & arr)
```

This function must be implemented iteratively. In a comment above your iterative implementation, briefly explain what the time complexity is and how you found it.

```
int max_subarray_sum_dnc(const std::vector<int> & arr)
```

Your divide and conquer algorithm must run in **$O(n \log n)$** . In a comment above your divide and conquer implementation, briefly explain why your function runs in $O(n \log n)$ time.

Testing

The `main()` function provided in `lab2.cpp` will generate randomized integer **vectors** and call both functions for each part of the lab. The results from the iterative and divide and conquer approaches will be compared, and if they agree, the tester will report that you passed the tests.

The test code in `lab2.cpp` is not very rigorous or complicated, so feel free to modify it if you wish. Make the vectors bigger, increase the number of tests, etc. C++ vectors are like Java ArrayLists. If you've never used them, check out the documentation below.

<https://cplusplus.com/reference/vector/vector/>

Submission

Submit your source file (**dnc.cpp**) on D2L. This lab must be submitted individually.